Überblick

1. Modulkatalog

1.1. Echtzeit Rendering

1.1.1. Rendering-Pipeline

1.1.2. Texturen

1.1.3. Schatten

1.2. Optimierung von 3D-Modellen

1. **Geometry**

- Aim for the lowest polygon count that preserves silhouette and motion-critical detail.
- Use retopology for animation-friendly topology; decimation/quadric simplification for non-deforming assets.
- Remove hidden/internal geometry and unused vertex attributes (colors, weights) before export.

2. **Normal/detail baking**

- Bake high-poly detail (sculpt) to normal, ambient occlusion, curvature and height maps on a low-poly mesh.
- Use cage/expanded rays to avoid projection errors; check smoothing/tangent-space consistency.

3. **LODs & culling**

- Generate multiple LODs (e.g., LOD0..LOD3) sized to your platform budgets and switching distances.
- Combine with frustum and occlusion culling to avoid rendering off-screen or occluded objects.

4. **UVs & texel denstiy**

- Pack UVs to maximize usable texture space; prioritize visible areas with higher texel density.
- Minimize seams in visible regions; reuse mirrored UVs for symmetric parts when appropriate.

5. **Textures & atlasing**

- Bake PBR maps (albedo/base color, normal, ORM/metallic-roughness, AO).
- Use texture atlases to combine small materials and reduce draw calls.
- Choose platform-appropriate resolutions (e.g., 2048/1024/512) and mipmap levels.

6. **Compression & file formats**

- Use glTF/glb for web/portable delivery; enable Draco mesh compression for geometry.
- Use GPU-native texture compression (ASTC, BC7, ETC2) to reduce GPU memory usage.
- Quantize vertex attributes where acceptable (positions, UVs, normals).

7. **Materials & draw cells**

- Consolidate materials and texture sets to reduce draw calls.
- Prefer simple PBR materials and avoid expensive shader features unless needed.

8. **Rigging & animation optimization**

- Limit bone counts per vertex; bake or retarget animations and remove unused animation channels.
- Use compression schemes (keyframe reduction, delta compression) for animation data.

9. **Pipeline & automation**

- Define budgets (tris, texture sizes, LOD count) per asset type and platform.
- Automate validation (triangle counts, missing maps, inverted normals) in CI or export scripts.
- Test on target hardware early and iterate.

10. **Web/AR/VR specifics**

- Prioritize file size and GPU memory; use smaller textures, aggressive LODs, and Draco compression.
- Profile on devices (mobile, headset) for frame time and memory peaks.

1.2.1. Tools & techniques (common)

- Modeling/retopology: Blender, Maya, 3ds Max, ZBrush (ZRemesher)
- Simplification: Blender Decimate, MeshLab, Instant Meshes, Simplygon
- Baking/texturing: Substance Painter, xNormal, Marmoset Toolbag
- Export/format: glTF (glb), FBX; Draco (mesh), KTX2/BCn/ASTC for textures
- Optimization suites: glTF-Pipeline, gltfpack, Blender add-ons, engine tools (Unity/Unreal LOD tools)

1.2.2. Quick actionable workflow (one-pass)

1. Sculpt high-res → retopologize low-res.
2. UV unwrap low-res with prioritized texel density.
3. Bake normal/AO/ORM maps from high to low.
4. Create LODs and atlas textures where feasible.
5. Compress textures to GPU format; apply Draco to meshes.
6. Import to engine, profile, and iterate.

1.2.3. Further reading/resources

- Microsoft — “Best practices for converting and optimizing real-time 3D objects” (guidelines, PBR, LODs, textures)
- Articles/tutorials on LOD, baking, and texture atlasing (game-dev focused sites such as Unity/Unreal docs)
- glTF specification and tools (glTF overview, gltfpack, glTF-Pipeline, Draco)
- Blender manual sections: Decimate, UV unwrapping, baking, and LOD generation
- Simplygon / Marmoset / Substance Painter docs for automation and baking workflows
- MeshLab and Instant Meshes for mesh cleaning and remeshing

Sources with links Microsoft — Best practices for converting and optimizing real-time 3D objects: \ https://learn.microsoft.com/en-us/dynamics365/mixed-reality/guides/3d-content-guidelines/best-practices

Microsoft — Optimize your 3D objects (convert/optimize for mixed reality / glTF): \ https://learn.microsoft.com/en-us/dynamics365/mixed-reality/guides/3d-content-guidelines/optimize-models

Microsoft — Overview of preparing 3D objects (glTF guidance): \ https://learn.microsoft.com/en-us/dynamics365/mixed-reality/guides/3d-content-guidelines/overview

Khronos Group — Art Pipeline for glTF (glTF authoring & PBR pipeline): \ https://www.khronos.org/blog/art-pipeline-for-gltf

Khronos Group — glTF (spec, tools, sample assets, validators): \ https://www.khronos.org/gltf

glTF Sample Models / Sample Assets repository (browse example assets): \ https://github.com/KhronosGroup/glTF-Sample-Models

glTF Validator / Tools overview (Khronos tools page): \ https://github.com/KhronosGroup/glTF-Validator

Draco mesh compression (Google): \ https://github.com/google/draco

gltfpack (optimization and packing tool): \ https://github.com/zeux/meshoptimizer/tree/master/gltf

Babylon.js Sandbox (quick glTF preview/testing): \ https://sandbox.babylonjs.com

three.js glTF examples & exporters (practical examples): \ https://threejs.org/examples/#webgl_loader_gltf

Blender — glTF export documentation (official exporter): \ https://docs.blender.org/manual/en/latest/addons/io_scene_gltf2.html

Substance (Adobe) — PBR / texture export and glTF workflows (Substance Painter docs): \ https://substance3d.adobe.com/documentation/spdoc/export-textures-188973989.html

Marmoset Toolbag — baking & previewing normal/AO maps: \ https://marmoset.co/toolbag/

Unity — Model import, LODs, mesh/texture optimization docs: \ https://docs.unity3d.com/Manual/OptimizingGraphicsPerformance.html

Unreal Engine — LODs, mesh simplification and Nanite guidance: \ https://docs.unrealengine.com/en-US/RenderingAndGraphics/Optimization/index.html

1.3. Globale Rendering Verfahren

1.3.1. Raytracing

1.3.2. Radiosity

1.4. Volume Rendering

1.5. Partikelsysteme

1.6. Methoden der Computeranimation

2. Grafikkarten und Hardware

3. Rendering-Verfahren

4. Modellierung

5. Animation

6. Licht und Schatten

7. Farbwissenschaft

8. Anwendungen der Computergrafik

9. Technische Aspekte

9.1. Grafikpipeline

The graphics pipeline is a sequence of stages that graphics data goes through to produce a final rendered image on the screen. This process transforms 3D objects into a 2D image, allowing for realistic visual representations in video games, simulations, and other graphical applications. Here's a breakdown of its key stages:

Index	Stage	Description
1.	Application Stage	The stage where the application prepares the scene, defining objects, cameras, and lights.
2.	Geometry Stage	Responsible for processing the vertices of 3D shapes. This includes transformations like scaling, rotation, and translation.
3.	Rasterization Stage	Converts the geometry into pixels on the screen. Determines which pixels should be drawn based on the shapes.
4.	Fragment Stage	Processes pixel data by determining color, lighting, and texture information. Each pixel's attributes are calculated here.
5.	Output Merger Stage	Combines various fragments into a final frame and applies effects like alpha blending and depth testing.

9.1.1. Application Stage

In the application stage, developers set up the scene, including defining the vertices of geometric shapes, cameras, and lights. This is where the application utilizes functions from graphics libraries to prepare objects for rendering.

9.1.2. Geometry Stage

This stage is focused on the transformation of vertices. It includes:

- **Model Transformation**: Positions objects in the world.
- **View Transformation**: Adjusts the scene from the camera's perspective.
- **Projection Transformation**: Converts 3D scene data into a 2D view.

9.1.3. Rasterization Stage

During rasterization, the 3D image representation is converted into a 2D grid of pixels. The graphics hardware determines which pixels correspond to the triangles formed during the geometry stage. This stage involves:

- **Triangulation**: Splitting complex shapes into triangles.
- **Pixel Coverage**: Deciding which pixels are covered by triangles.

9.1.4. Fragment Stage

In this stage, each pixel's color and other characteristics are computed. This includes:

- **Texture Mapping**: Applying images (textures) to surfaces.
- **Lighting Calculations**: Determining how light interacts with surfaces for realistic effects.

9.1.5. Output Merger Stage

Finally, the last stage combines the fragments from the previous stage into a complete image. This involves:

- **Depth Testing**: Ensures that closer objects obscure farther ones.
- **Blending**: Combines colors from multiple sources for effects like transparency.

9.2. Geometrische Darstellung

9.3. Textur- und Materialverarbeitung

9.4. Licht- und Schattenberechnung

9.5. Rendering-Techniken

9.6. Animationstechniken

9.7. Farbdarstellung und -verwaltung

9.8. Hardware und Software